Составные типы, функции, указатели

Занятие 2

Массивы

Объявление

```
var x [3]int                    // [0 0 0]
var x = [3]int{10, 20, 30}      // [10 20 30]
var x = [...]int{10, 20, 30}    // размер определяется автоматически
```

Важно: размер массива — часть типа! `[3]int` ≠ `[4]int`

Массивы

Особенности

- Фиксированный размер (нельзя изменить)
- Сравниваемы: `x == y` работает
- Копируются при передаче в функцию
- Редко используются напрямую

> Используйте массивы только для фиксированных данных (хеши, IP-адреса)

Срезы (Slices)

Основной тип для последовательностей

```go
var x []int                // nil-срез
var x = []int{10, 20, 30} // литерал
x := make([]int, 5)        // длина 5, емкость 5
x := make([]int, 0, 10)    // длина 0, емкость 10
```

Срезы: nil vs пустой

```go
var s1 []int       // nil-срез
s2 := []int{}      // пустой срез

fmt.Println(s1 == nil)  // true
fmt.Println(s2 == nil)  // false
fmt.Println(len(s1), len(s2))  // 0 0
```

> Рекомендация: используйте nil-срезы по умолчанию

Функция append

```
var x []int
x = append(x, 10)            // добавляем один элемент
x = append(x, 11, 12, 13)    // добавляем несколько
x = append(x, y...)           // добавляем другой срез
```

Правило: всегда присваивайте результат обратно!

Длина и емкость

```go
x := make([]int, 0, 5)
fmt.Println(len(x), cap(x))  // 0 5

x = append(x, 1, 2, 3)
fmt.Println(len(x), cap(x))  // 3 5

x = append(x, 4, 5, 6)       // превышение емкости
fmt.Println(len(x), cap(x))  // 6 10 (удвоилось)
```

Срезание срезов

```go
x := []string{"a", "b", "c", "d"}
y := x[:2]    // ["a", "b"]
z := x[1:]    // ["b", "c", "d"]
d := x[1:3]   // ["b", "c"]
e := x[:]     // копия всего среза
```

Общая память при срезании

```
x := []string{"a", "b", "c", "d"}
y := x[:2]
y = append(y, "z")   // изменяет x!

fmt.Println(x)  // [a b z d]
```

Проблема: подсрез использует ту же память!

Функция copy

```go
x := []int{1, 2, 3, 4}
y := make([]int, 4)
n := copy(y, x)     // копирует в y

// Копирование подмножества
z := make([]int, 2)
copy(z, x[2:])      // [3 4]
```

Ваши вопросы?

Задание

Время: 10 минут

1. Создайте пустой срез `var x []int`
2. Добавляйте в него элементы по одному с помощью `append`
3. После каждого добавления выводите длину (`len`) и ёмкость (`cap`) среза
4. Понаблюдайте, как меняется ёмкость

Отображения (Maps)

Объявление

```go
var m map[string]int        // nil-отображение (нельзя писать!)
m := map[string]int{}       // пустое, можно писать
m := make(map[string]int, 10)  // с начальной емкостью

// Литерал
teams := map[string][]string{
    "Orcas": {"Fred", "Ralph"},
    "Lions": {"Sarah", "Peter"},
}
```

Отображения: работа

```go
m := map[string]int{}
m["key"] = 10          // запись
v := m["key"]          // чтение: 10
v := m["missing"]      // 0 (ключа нет — нулевое значение)

// Проверка существования
v, ok := m["key"]      // ok = true если ключ есть
delete(m, "key")       // удаление
clear(m)               // очистка (Go 1.21+)
```

Идиома "запятая-ok"

```go
m := map[string]int{"hello": 5, "world": 0}

v, ok := m["hello"]    // v = 5, ok = true
v, ok := m["world"]    // v = 0, ok = true (ключ есть!)
v, ok := m["missing"]  // v = 0, ok = false
```

> Используйте `ok` для различения "ключа нет" и "ключ = 0"

Мар как множество

```go
// struct{} не занимает память (0 байт)
set := map[int]struct{}{}
vals := []int{5, 10, 2, 5, 8, 7, 3}
for _, v := range vals {
    set[v] = struct{}{}
}
fmt.Println(len(set))  // 6 (дубликат 5 удален)

// Проверка наличия
if _, exists := set[5]; exists {
    fmt.Println("5 есть в множестве")
}
```

Ваши вопросы?

Задание

Время: 10 минут

Дана строка. Выведите на экран частоту каждого символа:

```
s := "hello world"
// Вывод:
// h: 1
// e: 1
// l: 3
// o: 2
// ...
```

Функции

Объявление функций

```
func div(num, denom int) int {
    if denom == 0 {
        return 0
    }
    return num / denom
}
```

- Нет именованных параметров
- Нет опциональных параметров
- Всегда нужно передавать все аргументы

Вариативные параметры

```go
func addTo(base int, vals ...int) []int {
    out := make([]int, 0, len(vals))
    for _, v := range vals {
        out = append(out, base+v)
    }
    return out
}

addTo(3)                // []
addTo(3, 2)             // [5]
addTo(3, 2, 4, 6)       // [5 7 9 11]
addTo(3, []int{1,2}...) // [4 5]
```

Множественные возвращаемые значения

```
func divAndRemainder(num, denom int) (int, int, error) {
    if denom == 0 {
        return 0, 0, errors.New("division by zero")
    }
    return num / denom, num % denom, nil
}

result, remainder, err := divAndRemainder(5, 2)
result, _, _ := divAndRemainder(5, 2)  // игнорируем часть
```

Именованные возвращаемые значения

```go
func divAndRemainder(num, denom int) (result int, remainder int, err error) {
    if denom == 0 {
        err = errors.New("division by zero")
        return  // пустой return
    }
    result, remainder = num/denom, num%denom
    return
}
```

> Осторожно с пустыми return — могут запутывать!

Функции как значения

```go
var myFunc func(string) int

func f1(a string) int { return len(a) }
func f2(a string) int { return len(a) * 2 }

myFunc = f1
fmt.Println(myFunc("hello"))  // 5

myFunc = f2
fmt.Println(myFunc("hello"))  // 10
```

Функциональные типы

```go
type opFunc func(int, int) int

var opMap = map[string]opFunc{
    "+": func(a, b int) int { return a + b },
    "-": func(a, b int) int { return a - b },
    "*": func(a, b int) int { return a * b },
    "/": func(a, b int) int { return a / b },
}

result := opMap["+"](2, 3)  // 5
```

Анонимные функции и замыкания

```go
// Анонимная функция
f := func(j int) {
    fmt.Println("value:", j)
}
f(5)

// Замыкание
a := 20
f := func() {
    a = 30  // изменяет внешнюю переменную
}
f()
fmt.Println(a)  // 30
```

Возврат функций

```go
func makeMult(base int) func(int) int {
    return func(factor int) int {
        return base * factor
    }
}

double := makeMult(2)
triple := makeMult(3)

fmt.Println(double(5))  // 10
fmt.Println(triple(5))  // 15
```

Ваши вопросы?

Задание

Время: 10 минут

Напишите функцию `mapInts`, которая принимает срез чисел и функцию-преобразователь, применяет функцию к каждому элементу и возвращает новый срез:

```go
func mapInts(nums []int, f func(int) int) []int {
    // ваша реализация
}

nums := []int{1, 2, 3, 4, 5}
doubled := mapInts(nums, func(x int) int { return x * 2 })
fmt.Println(doubled)  // [2 4 6 8 10]
```

Указатели

Основы указателей

```go
var x int = 10
var p *int = &x     // & — взятие адреса

fmt.Println(p)      // 0xc000014090 (адрес)
fmt.Println(*p)     // 10 (разыменование)

*p = 20             // изменение через указатель
fmt.Println(x)      // 20
```

Важно: В Go нет арифметики указателей! (в отличие от C/C++)

Всегда передаём по значению

Go **всегда** передаёт параметры по значению (создаёт копию):

```go
func modify(x int) {
    x = 100  // изменяет копию
}

func modifyPtr(x *int) {
    *x = 100  // изменяет оригинал через указатель
}

a := 10
modify(a)       // a = 10 (не изменилось)
modifyPtr(&a)   // a = 100 (изменилось!)
```

Проверка на nil

Всегда проверяйте указатель перед разыменованием!

```
func process(p *int) {
    if p == nil {
        fmt.Println("nil pointer")
        return
    }
    fmt.Println(*p)  // безопасно
}

var p *int  // nil
process(p)  // не упадёт
```

Разыменование `nil` вызывает **панику**!

Создание указателей

```go
// Из существующей переменной
x := 10
p := &x

// Функция new
p := new(int)  // указатель на int с нулевым значением
*p = 20

// Нельзя взять адрес константы!
// p := &10  // ОШИБКА
```

Указатели и функции

```go
func failedUpdate(px *int) {
    x := 20
    px = &x  // меняет копию указателя
}

func successUpdate(px *int) {
    *px = 20  // меняет значение по адресу
}

x := 10
failedUpdate(&x)   // x = 10
successUpdate(&x)  // x = 20
```

Срез — структура с указателем

Срез внутри содержит 3 поля:

- указатель на массив данных
- длина (len)
- ёмкость (cap)

```
func modifySlice(s []int) {
    s[0] = 100        // OK: изменяет данные по указателю
    s = append(s, 1) // НЕ изменяет оригинал! (новая структура)
}
```

Map — тоже содержит указатель

```go
func modifyMap(m map[int]string) {
    m[1] = "new"      // OK: изменяет оригинал
    delete(m, 2)      // OK: удаляет из оригинала
}
```

Map и Slice передаются по значению, но содержат указатель на данные.

Когда использовать указатели

- Для изменения значения в функции
- Для обозначения "значение не задано" (nil)
- Для больших структур (оптимизация)
- Для циклических структур данных

Когда НЕ использовать указатели

- Для маленьких структур и простых типов
- Для полей, которые не должны быть nil
- "На всякий случай" — без реальной причины

> Указатели увеличивают нагрузку на GC

Безопасность возврата указателя

```go
func createPointer() *int {
    x := 10
    return &x  // OK! x "убегает" в кучу
}

p := createPointer()
fmt.Println(*p)  // 10
```

В Go безопасно возвращать указатель на локальную переменную!

Ваши вопросы?

Задание

Время: 10 минут

Напишите функцию `sortTwo(a, b *int)`, которая сортирует значения двух переменных так, чтобы в `*a` было меньшее значение, а в `*b` — большее.

```
x, y := 5, 3
sortTwo(&x, &y)
fmt.Println(x, y)  // 3 5

x, y := 10, 20
sortTwo(&x, &y)
fmt.Println(x, y)  // 10 20 (без изменений)
```

Пользовательские типы и алиасы

Объявление новых типов

```go
type Score int                      // новый тип на основе int
type Converter func(string) Score // функциональный тип
type TeamScores map[string]Score  // составной тип
```

Зачем?

- Семантическое различие (Celsius ≠ Fahrenheit)
- Улучшение читаемости
- Возможность добавления методов (позже)

Разные типы — даже с одинаковым базовым

```go
type Celsius float64    // новый тип
type Fahrenheit float64 // другой новый тип

var c Celsius
var f Fahrenheit

// c = f                // ОШИБКА: разные типы
// c == f               // ОШИБКА: нельзя сравнить

fmt.Println(c == 0)               // OK
fmt.Println(c == Celsius(f))      // OK — явное преобразование
```

Алиасы типов

```go
type byte = uint8        // алиас (символ =)
type rune = int32        // алиас

type MyInt = int         // MyInt и int — ОДИН И ТОТ ЖЕ тип

var x int = 10
var y MyInt = x          // OK! Это один тип
fmt.Println(x == y)      // OK! Можно сравнивать
```

Отличие:

- `type MyInt int` — **новый тип**, отличный от int
- `type MyInt = int` — **алиас**, тот же самый тип

Алиасы: когда использовать?

```go
// Алиас — для удобства и совместимости
type Handle = int           // просто другое имя
type Callback = func() error // алиас для сигнатуры функции

// Новый тип — для семантического различия
type Celsius float64        // нельзя спутать с Fahrenheit
type UserID int             // нельзя спутать с OrderID
```

Пустой интерфейс (any)

```go
var x any = 10
x = "hello"
x = []int{1, 2, 3}

// Type assertion
s, ok := x.(string)
if ok {
    fmt.Println(s)
}
```

`any` — это алиас для `interface{}`, принимает любой тип.

Type switch

```go
func printType(v any) {
    switch v := v.(type) {
    case int:
        fmt.Println("int:", v)
    case string:
        fmt.Println("string:", v)
    default:
        fmt.Println("unknown")
    }
}
```

Ваши вопросы?

Задание

Время: 10 минут

1. Создайте типы `Celsius` и `Fahrenheit` на основе `float64`
2. Напишите функцию `CelsiusToFahrenheit(c Celsius) Fahrenheit`
   - Формула: `F = C * 9/5 + 32`
3. Напишите функцию `FahrenheitToCelsius(f Fahrenheit) Celsius`
   - Формула: `C = (F - 32) * 5/9`
4. Проверьте: `CelsiusToFahrenheit(0)` должно вернуть `32`

Сегодня мы узнали

- **Массивы** — фиксированный размер, часть типа, редко используются напрямую
- **Срезы** — основной тип для последовательностей, len/cap, append, срезание
- **Мар** — ключ-значение, идиома "запятая-ok", nil-отображение
- **Функции** — вариативные параметры, множественный возврат, именованные значения
- **Замыкания** — функции как значения, анонимные функции, возврат функций
- **Указатели** — &, *, nil, передача по значению, когда использовать
- **Типы и алиасы** — новый тип vs алиас (=), type switch